

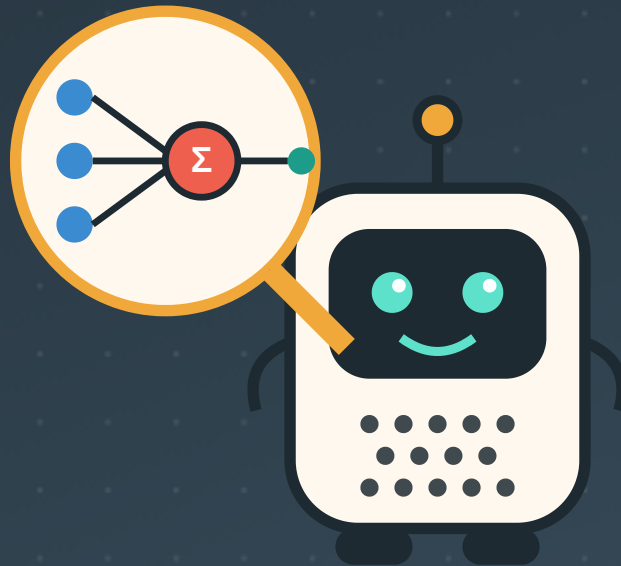
DEEP LEARNING, TOLD SIMPLY

Book 2 of 5

One Tiny Brain Cell

Inside Nova's head is a machine so simple you can run it with a pencil. We will teach it logic by hand — and then hand it the one puzzle it can never learn.

Inside: the real neuron and its cartoon · connectionism, or why knowledge lives in the wires · the perceptron · Rosenblatt's learning rule, worked by hand for NOT, AND and OR · linearly separable data · the XOR wall, with a four-line proof · a perceptron in twenty lines of Python



Deep Learning (DNN) · a five-book story

Prof. Saurabh

Where we are in the story

LAST TIME

Asha wanted her robot Nova to wake up when it hears “Hey Nova”. Nobody can write that program by hand, because we do not know the rule ourselves. So we learned to **show instead of tell**: collect examples, build a machine with **knobs**, and turn the knobs until the answers come out right.

Every such project has four ingredients — **data, model, objective, optimizer** — and learns by one loop: **guess, measure, nudge, repeat**. A deep model is a stack of layers, and the layers are made of tiny units called *neurons*. We never opened one. Today we do.

THIS TIME

We take one neuron out of Nova’s head and look at it under a magnifying glass. It is embarrassingly simple: add up some inputs, each multiplied by a knob, and fire if the total is big enough. That is a **perceptron**.

Then we do something no textbook lets you do enough: we **teach it with a pencil**. We watch every knob move as it learns NOT, AND and OR. We find out what it can learn — anything you can cut with one straight line — and we meet XOR, a four-example puzzle it can never learn, which will haunt us until Book 5.



What you need

A pencil, paper, and the ability to multiply and add small whole numbers. That is all. Every calculation in this book uses the numbers 0, 1, 2 and 3. If you can also open the companion notebook (Book2_Perceptron_From_Scratch.ipynb), you can check every line of your paper work against a computer — and you should.

The boxes you will meet

Same as Book 1. **Story** (Asha and Nova) · **Big idea** (the one thing to remember) · **Think of it like...** (an everyday picture) · **Stop and think** (pause and try before reading on) · **Our answer** · **Try it yourself** (paper or notebook) · **Careful** (a common confusion) · **Spark** (history and wonder) · **Words to know**.

1

Inside your head: the real neuron

Before we build a fake brain cell, let's look at a real one. It is a tiny voter that listens to its neighbours.

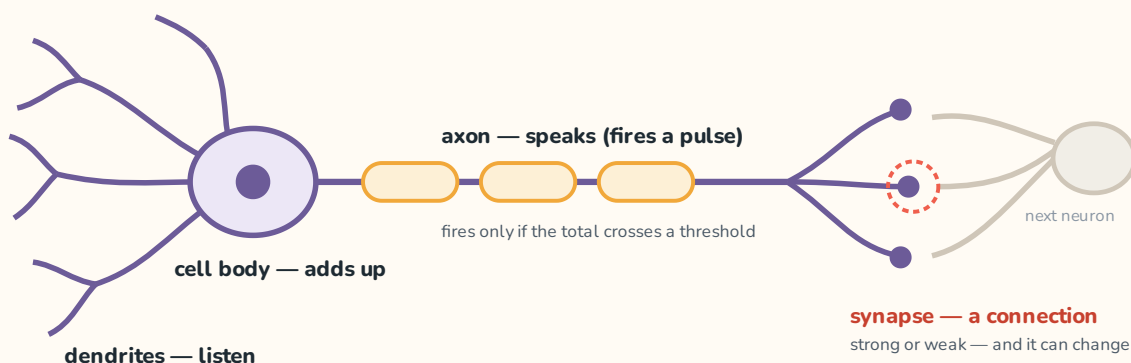
STORY

Asha's cousin Meera is studying medicine. Asha shows her Nova and says, "It has layers made of neurons." Meera laughs. "*Neurons?* I dissected one last week. It had nothing to do with numbers." Asha thinks about it and admits: she has no idea what a real neuron actually does. So they look at one together.

What a real neuron does, in one breath

Your brain has about 86 billion neurons. Each one is a cell with three parts you need to know. **Dendrites** are branches that *listen* — they receive signals from thousands of other neurons. The **cell body** *adds up* everything the dendrites bring in. The **axon** is a long cable that *speaks* — if the total in the cell body crosses a threshold, the neuron fires an electrical pulse down the axon to the next neurons.

The place where one neuron's axon meets another neuron's dendrite is called a **synapse**. A synapse can be **strong** (its signal counts a lot) or **weak** (its signal barely matters). And here is the fact that makes everything in this course possible: **synapses change their strength with experience**. Learning, in your head, is mostly synapses getting stronger or weaker.



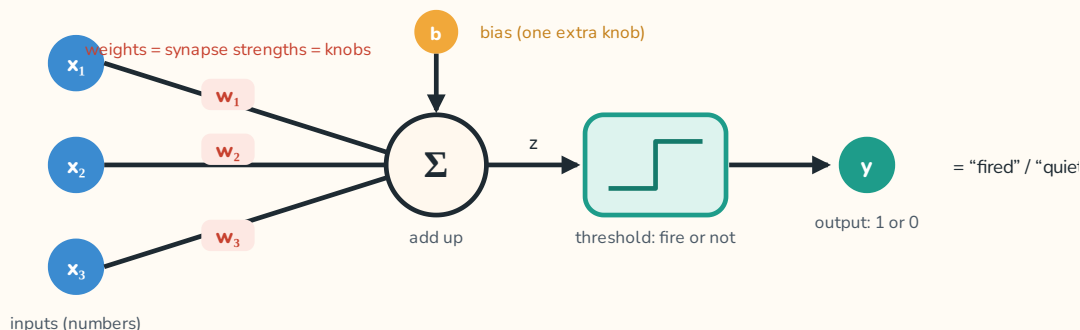
A neuron, simplified. Signals arrive at the dendrites through synapses of different strengths, are added up in the cell body, and — if the total is big enough — a pulse leaves down the axon. Real neurons are messier, but this is the part that matters for us.

THINK OF IT LIKE... A COMMITTEE VOTE

A neuron is a committee member who listens to thousands of colleagues before deciding. Some colleagues she trusts a lot (strong synapses) — their "yes" counts heavily. Some she barely listens to (weak synapses). A few she treats as warnings — their "yes" counts *against*. She adds it all up in her head. If the total is big enough, she votes yes and tells the next committee. Over years, she learns whom to trust: some voices grow louder in her head, some fade. That is learning.

The cartoon: an artificial neuron

In 1943, Warren McCulloch (a neuroscientist) and Walter Pitts (a logician, then only 20) asked a bold question: *what if we keep only the arithmetic and throw away the biology?* Keep “add up the inputs, each counted by its connection strength, and fire if the total is big enough”. Throw away the chemistry, the shape, the timing. What is left is a rule you can write in one line:



The artificial neuron. Inputs come in as numbers. Each is multiplied by its weight (a knob). Everything is added, plus a bias. If the total z is big enough, the output is 1; otherwise 0. Compare it with the real neuron above, part by part.

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b \quad \hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

the weighted sum, then the threshold. In shorthand, $z = \mathbf{w} \cdot \mathbf{x} + b$: the dot product of the weights with the inputs, plus the bias.

Real neuron	Artificial neuron	What it means for us
dendrites receive signals	inputs $x_1, x_2, x_3 \dots$	the numbers we feed in (from Book 1: a vector)
synapse strength, strong or weak, positive or inhibiting	weight $w_1, w_2, w_3 \dots$ (any number, + or -)	the knobs — how much, and in which direction, each input matters
cell body adds everything up	$z = \sum_i w_i x_i + b$	one multiplication per input, then a sum
fires if the total crosses a threshold	output 1 if $z \geq 0$, else 0	the decision; the bias b sets how easy it is to fire
synapses change with experience	weights change with training	learning = changing the weights (Chapter 4)

⚠ CAREFUL — A CARTOON, NOT A COPY

An artificial neuron is to a real neuron what a stick figure is to a person. Real neurons use timing, chemistry and feedback that the cartoon ignores. We are *not* building a brain. We borrowed one idea — **weighted sum, then threshold, and the weights can learn** — and it turned out to be enough. Meera is right that it has little to do with biology. Asha is right that the idea came from biology. Both are true.

Many simple things, connected

Connectionism is one idea: knowledge does not live in rules. It lives in the strengths of the connections between many simple units.

STOP AND THINK

You can recognise your best friend's face in a crowd, in bad light, from the side. *Where* in your head is that ability stored? Is there a rule somewhere — “if the nose is this shape and the eyes are that far apart, then...”? Could a surgeon find it and cut it out?

✓ OUR ANSWER

There is no rule and no single place. The ability is spread across millions of synapses — a *pattern* of strong and weak connections. No single neuron knows your friend. The pattern does. That is why you could not write the rule down in Book 1: there was never a rule to find, only a pattern of connection strengths.

Two ways to build a mind

The rulebook way

Intelligence is **rules on symbols**. A human writes the rules: “if it has three sound bursts, wake up.” Knowledge lives in the rulebook. Remove one rule and that piece of knowledge is simply gone. This was Asha's 400 rules — and most of AI from the 1950s to the 1980s.

The connectionist way

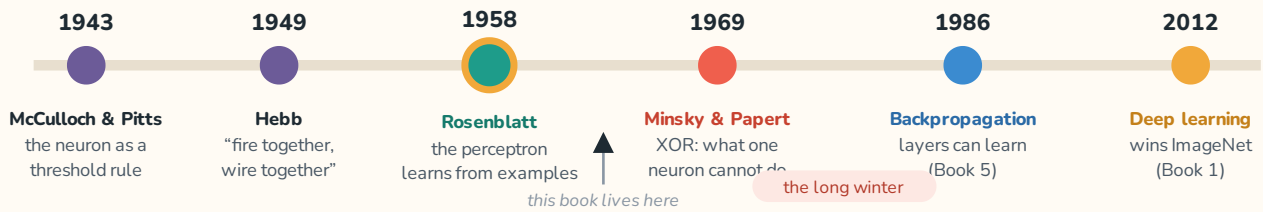
Intelligence is **many simple units, richly connected**. Knowledge lives in the *strengths* of the connections — the weights. No unit knows anything on its own. Learning means changing the weights. Remove a few connections and the knowledge fades a little but does not vanish.

IN ONE LINE **Connectionism:** the program *is* the weights. Nova's whole ability to hear its name is nothing but a long list of numbers on its connections.

THINK OF IT LIKE... A VILLAGE THAT KNOWS A RUMOUR

Nobody in the village knows the whole story. Each person knows a piece and trusts certain neighbours more than others. Ask the village a question and the answer emerges from who tells whom, how loudly, and who believes it. If two villagers move away, the rumour survives — slightly changed. Now a rulebook: it lives in one book, on one shelf. Tear out a page and that fact is gone forever. Brains are villages, not books. So are deep networks.

A short history, with three sparks



Seventy years in one line. The idea was born in 1943, learned to learn in 1958, was declared dead in 1969, came back with layers in 1986, and took over the world in 2012. Books 2 to 5 walk this line.

✧ SPARK 1 — HEBB'S RULE (1949)

Donald Hebb, a psychologist, proposed the first learning rule ever written for neurons: *when two neurons fire at the same time, the connection between them gets stronger*. "Cells that fire together wire together." Notice what kind of rule it is — a rule for turning a knob. It is the great-grandparent of every learning rule in this course.

✧ SPARK 2 — ROSENBLATT'S MACHINE (1958)

Frank Rosenblatt built the **Mark I Perceptron**: a camera of 400 light sensors wired to units whose weights were actual motor-driven knobs. It learned to tell simple shapes apart *from examples* — the first machine to do so. The newspapers went wild: one reported that the navy expected it to soon "walk, talk, see, write, reproduce itself and be conscious of its existence." It could not. But something in it was true, and this book is about that something.

✧ SPARK 3 — THE BOOK THAT STARTED A WINTER (1969)

Marvin Minsky and Seymour Papert wrote a careful book called *Perceptrons*. It proved, with mathematics, exactly what a single perceptron can and cannot learn. The most famous "cannot" is XOR — you will prove it yourself in Chapter 6. The field took the book as a death sentence. Funding dried up for nearly two decades. The fix (layers, plus a way to train them) was already half-known, but almost nobody looked.

✧ SPARK — SLOW CELLS, FAST BRAIN

A real neuron fires at most a few hundred times a second — millions of times slower than a transistor. Yet you recognise a friend's face in a tenth of a second. The brain wins not by speed but by **numbers and parallelism**: 86 billion slow neurons, thousands of connections each, all working at once. That is the design deep learning borrows.

📖 WORDS TO KNOW

Connectionism — the view that intelligence comes from many simple connected units, with knowledge stored in connection strengths. **Unit / neuron** — one simple element: weighted sum, then threshold. **Weight** — the strength of a connection; the knob. **Distributed representation** — knowledge spread over many weights, none of which holds it alone.

The perceptron: a tiny voting machine

One neuron, on its own, making one decision. Let's run it by hand and see what it really is: a straight wall with a side.

A **perceptron** is the artificial neuron from Chapter 1 used as a decision-maker. Here it is again, as a recipe you can run with a pencil:

THE PERCEPTRON, STEP BY STEP

- 1 · Multiply each input by its weight. 2 · Add the results, and add the bias. Call the total z . 3 · If $z \geq 0$, output 1. Otherwise output 0.

Should Asha go out tonight?

Let's make the weights mean something. Asha decides whether to go out using three yes/no facts: is it raining (x_1), are friends going (x_2), does she have money (x_3)? Yes = 1, no = 0. Her personal weights and bias:

$$z = (-3)x_1 + (+2)x_2 + (+1)x_3 + (-2) \longrightarrow \text{go out if } z \geq 0$$

raining? x_1	friends? x_2	money? x_3	working	z	decision
0	1	1	$0 + 2 + 1 - 2$	1	go out
1	1	1	$-3 + 2 + 1 - 2$	-2	stay in — rain outweighs everything
0	1	0	$0 + 2 + 0 - 2$	0	go out — friends alone are just enough
0	0	1	$0 + 0 + 1 - 2$	-1	stay in — money alone is not enough
0	0	0	$0 + 0 + 0 - 2$	-2	stay in — the default mood

The same five evenings, all at once. A computer does not fill the table row by row. It stacks the five evenings as the rows of a matrix $\mathbf{X}$, puts the three weights in a column $\mathbf{w}$, and one product gives every total:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -3 \\ 2 \\ 1 \end{bmatrix} + (-2) = \begin{bmatrix} 3 \\ 0 \\ 2 \\ 1 \\ 0 \end{bmatrix} - 2 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \\ -2 \end{bmatrix}$$

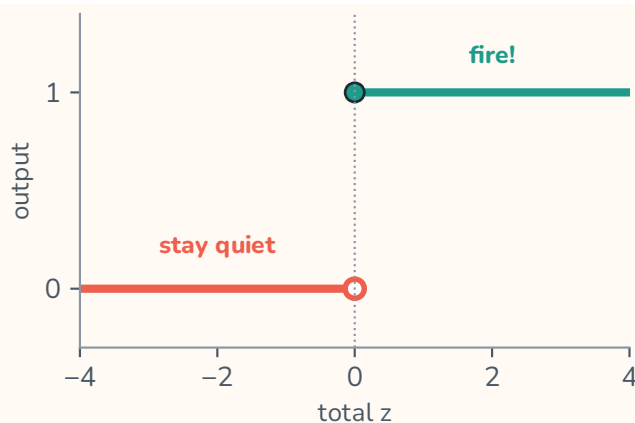
each row of the answer is one row of $\mathbf{X}$ dotted with $\mathbf{w}$ — exactly the “working” column of the table. Rows are examples, the column is the neuron's knobs. Every book from here on writes its maths this way too.

Now read the weights like a personality. **The sign** says *for* or *against*: rain is against (-3), friends are for ($+2$). **The size** says *how much it matters*: rain matters more than money. **The bias** is the default mood: with $b = -2$, Asha stays in unless the inputs add up to at least 2 points of “yes”. A cheerful Asha would have $b = 0$ and go out on far less.

∞ THINK OF IT LIKE... A BOUNCER WITH A POINTS CARD

Each guest earns or loses points for each thing about them (a weight per feature). The bouncer adds the points and lets you in only if you clear the bar. The bias is where the bar sits. That is the whole perceptron: **points, sum, bar.**

Two ways to say the same thing: “fire if $z = \sum_i w_i x_i + b \geq 0$ ” or “fire if $\sum_i w_i x_i \geq \theta$ ” where the **threshold** $\theta = -b$. For Asha, $\theta = 2$: “I need two points of yes.” Older books use θ ; we use b . They are the same knob with the sign flipped.

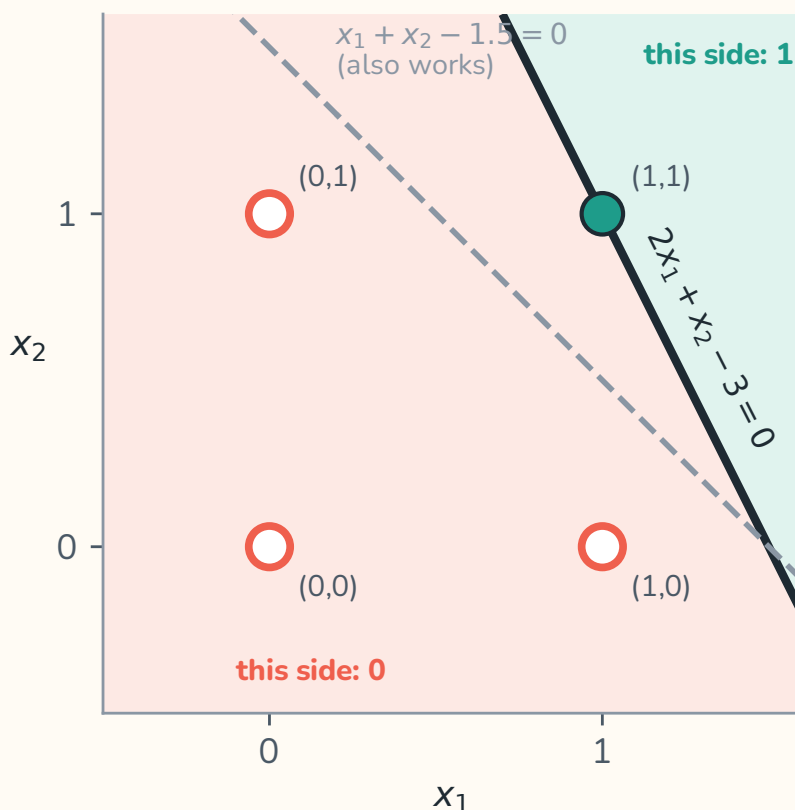


The step function. The perceptron’s decision is all-or-nothing: below zero, output 0; at zero or above, output 1. No “almost”. Remember this harshness — it will matter in Book 3.

The picture: a perceptron is a straight wall with a side

Take a perceptron with two inputs, x_1 and x_2 , so we can draw it. The total is $z = w_1 x_1 + w_2 x_2 + b$. Ask: *where is z exactly zero?* The answer is all the points with $w_1 x_1 + w_2 x_2 + b = 0$ — and from school algebra, that is the equation of a **straight line**.

On one side of that line, z is positive, so the perceptron says 1. On the other side, z is negative, so it says 0. That is everything a perceptron can do: it draws one straight line through the world and answers “which side are you on?”



A line and a side. The perceptron with $w = (2, 1)$, $b = -3$ fires exactly on the teal side of the line $2x_1 + x_2 - 3 = 0$. Here the four corner points are labelled the way the AND gate wants: only (1,1) gets a 1. The dashed line is a *different* perceptron that makes the same four decisions — many lines can do the same job.

💡 STOP AND THINK

(a) If you change only the bias b , what happens to the line? (b) If you double *all* three numbers — w_1 , w_2 and b — what happens to the decisions?

✓ OUR ANSWER

(a) The line **slides** sideways without turning — the bias sets *where* the wall stands, the weights set its *direction*. (b) Nothing changes: z just doubles, and a positive number doubled is still positive. The line is the same line. This is why the perceptron has many equally good solutions.

With three inputs the wall is a flat plane in 3-D space. With 16,000 inputs — Nova's clip — it is a flat wall in a 16,000-dimensional space that nobody can picture. Mathematicians call it a **hyperplane**. Do not let the word scare you: it is still a straight wall, and the perceptron still only asks "which side?"

🧪 TRY IT YOURSELF — DESIGN A PERCEPTRON

Nova hears three yes/no things in a clip: heard "Hey" (x_1), heard "Nova" (x_2), loud background noise (x_3). Choose weights and a bias so that Nova wakes up **only** when both words are heard **and** it is not too noisy. Check all eight input combinations on paper. (One answer, upside down:

$z = -1.1x_1 - 1.1x_2 + 0.0x_3 - 1$ check $(1, 1, 0) \rightarrow 1$, $(1, 0, 1) \rightarrow 0$, $(0, 1, 1) \rightarrow 0$, $(0, 0, 1) \rightarrow 0$, $(1, 0, 0) \rightarrow 0$, $(0, 1, 0) \rightarrow 0$, $(0, 0, 0) \rightarrow 0$, $(1, 1, 1) \rightarrow 1$.) Then a harder one: can you make Nova wake up when it hears *exactly one* of the two words, but not both? Keep a note of what happens — we return to this in Chapter 6.

4

Teaching it by hand: the learning rule

So far we chose the weights. Now the perceptron will choose them itself — and we will watch every single move.

STORY

Asha sets Nova a tiny homework: learn the AND gate. Four examples, two inputs, one answer each. “I won’t tell you the weights,” she says. “I’ll only tell you when you’re wrong.” Nova starts with every knob at zero — it knows nothing — and Asha reads out the examples one by one.

In 1958 Frank Rosenblatt wrote down what Nova should do each time Asha says “wrong”. It is the simplest learning rule that works, and it is the ancestor of everything in Books 3 to 5.

ROSENBLATT'S LEARNING RULE (ONE EXAMPLE AT A TIME)

- 1 $z = w_1x_1 + w_2x_2 + b, \quad \hat{y} = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$ **Guess.** Compute the total, then the output.
- 2 $e = y - \hat{y} \in \{+1, -1, 0\}$ **Measure.** +1: said 0, should be 1. -1: said 1, should be 0. 0: correct.
- 3 $w_i \leftarrow w_i + e x_i$ (every input i), $b \leftarrow b + e$ **Nudge.** Error $\times$ input, added to each knob; the bias's “input” is always 1.
- 4 $\mathbf{w} \leftarrow \mathbf{w} + e \mathbf{x}$ **The same nudge, all inputs at once** — the vector form; this is the one line of code in Chapter 7.

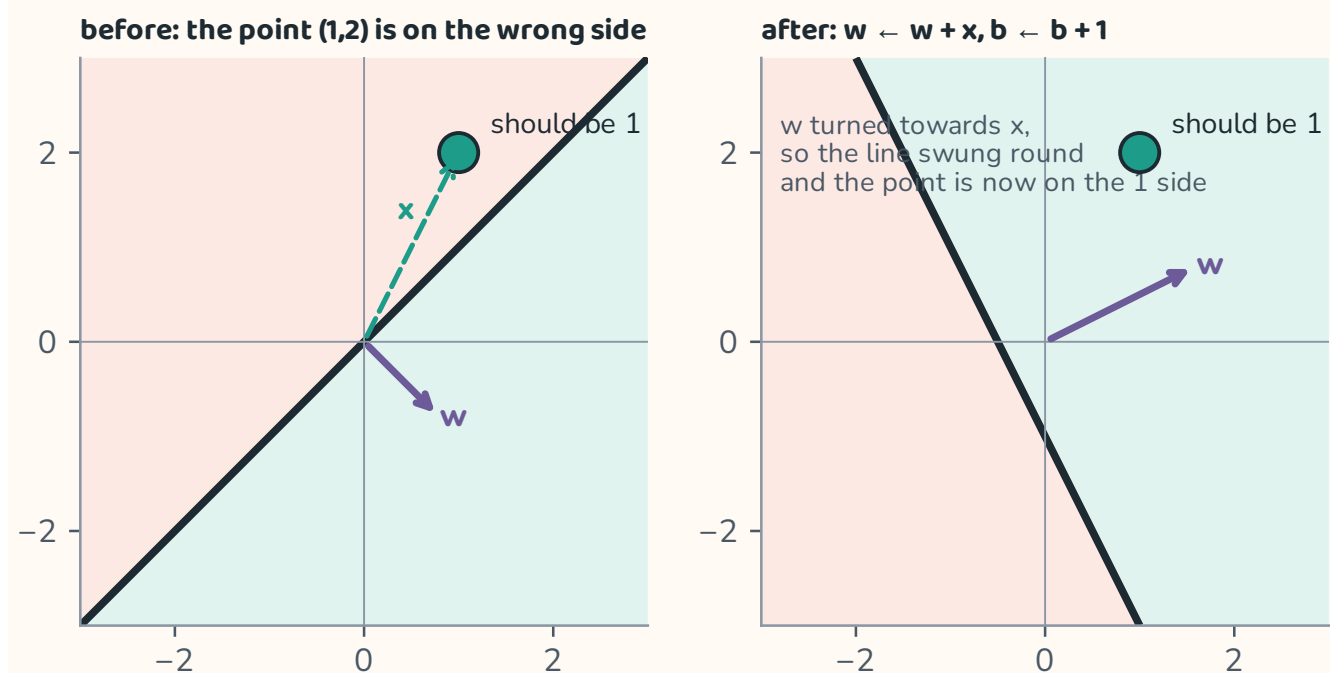
Then **repeat** with the next example. Go round all the examples again and again. Stop when a whole round has no mistakes.

(The full rule has a **learning rate** η multiplying the nudge: $w_i \leftarrow w_i + \eta e x_i$. We use $\eta = 1$ so the numbers stay small whole numbers. Starting from all-zero knobs, η only scales everything and does not change a single decision — try it in the notebook.)

Why the rule makes sense

What happened	error	The nudge	In words
correct	0	nothing changes	Don't fix what isn't broken.
said 0, should be 1	+1	$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x},$ $b \leftarrow b + 1$	z was too small. Make it bigger <i>for inputs like this one</i> : raise the knobs on the inputs that were on, and raise the bias.
said 1, should be 0	-1	$\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x},$ $b \leftarrow b - 1$	z was too big. Make it smaller for inputs like this one: lower the knobs on the inputs that were on, and lower the bias.

Notice the multiplication by x_i . If an input was **0**, it contributed nothing to z , so it gets no blame and no credit: its knob does not move. Only the wires that were **on** get adjusted. That is Hebb's idea from Chapter 2 with an error signal attached: *wires that were active during a mistake get corrected*.



What one nudge does to the wall. The arrow $\mathbf{w}$ always points to the “1” side of the line. A point that should be 1 sits on the wrong side. Adding $\mathbf{x}$ to $\mathbf{w}$ turns the arrow towards the point, so the wall swings round and the point ends up on the right side. One update, one swing. (Here the point is $(1, 2)$: before, $z = 1 \cdot 1 + (-1) \cdot 2 + 0 = -1 \rightarrow 0$. After, $z = 2 \cdot 1 + 1 \cdot 2 + 1 = 5 \rightarrow 1$.)

∞ THINK OF IT LIKE... THE LEARNING LOOP FROM BOOK 1

Guess (compute $\hat{y}$). Measure ($e = y - \hat{y}$). Nudge ($\mathbf{w} \leftarrow \mathbf{w} + e \mathbf{x}$). Repeat. It is exactly the loop from Book 1, Chapter 4, with the smallest possible model and the simplest possible nudge. Everything later in the course is this loop with a better model and a smarter nudge.

Warm-up: NOT, by hand

One input, two examples: NOT 0 = 1 and NOT 1 = 0. Start with $\mathbf{w} = 0$, $\mathbf{b} = 0$. Read every row; each row is one guess, one measurement, and maybe one nudge.

epoch	x	y	$z = (\text{weights} \cdot \text{inputs}) + \text{bias}$	guess	error	w after	b after
1	0	1	$0 \cdot 0 + 0 = 0$	1	0 ✓	0	0
1	1	0	$0 \cdot 1 + 0 = 0$	1	-1	-1	-1
2	0	1	$(-1) \cdot 0 + (-1) = -1$	0	+1	-1	0
2	1	0	$(-1) \cdot 1 + 0 = -1$	0	0 ✓	-1	0
3	0	1	$(-1) \cdot 0 + 0 = 0$	1	0 ✓	-1	0
3	1	0	$(-1) \cdot 1 + 0 = -1$	0	0 ✓	-1	0

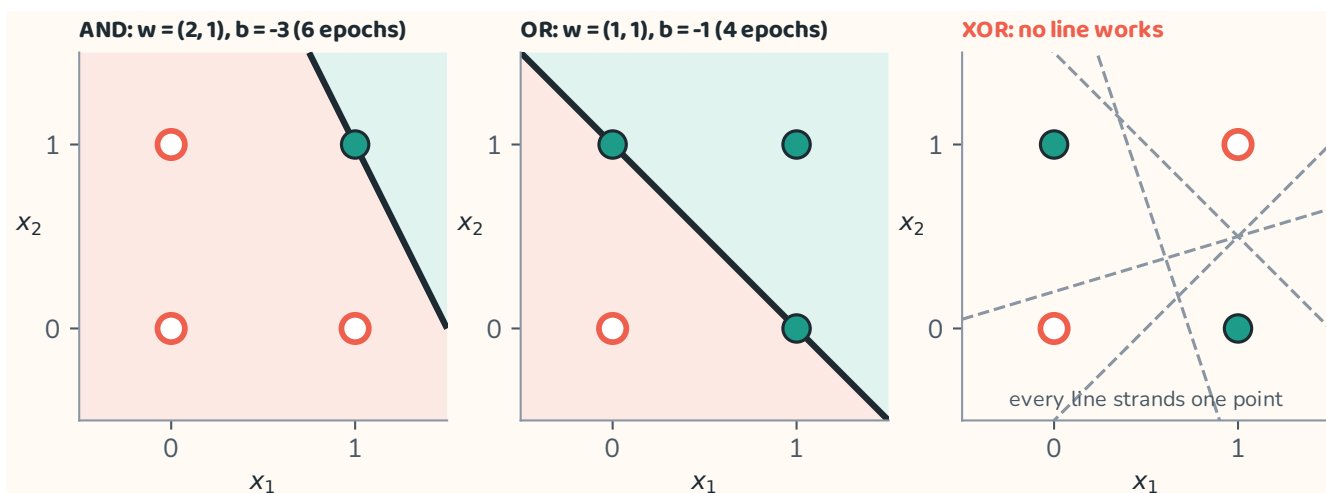
After epoch 3 there are no mistakes, so we stop. The perceptron has learned $\mathbf{w} = -1$, $\mathbf{b} = 0$: $z = -x$, which is ≥ 0 only when $x = 0$. That is NOT. (Check epoch 2, row 1: the earlier nudge had pushed the bias too low, so the input 0 was now wrongly answered; the rule pushed the bias back up. Two steps forward, one step back — normal.)

The main event: AND, by hand

Two inputs, four examples, start from $w_1 = w_2 = 0$, $b = 0$. This is the full trace — 24 guesses over 6 epochs. Follow at least the first two epochs line by line with your own pencil; after that, follow the pattern.

epoch	x	y	$z = (\text{weights} \cdot \text{inputs}) + \text{bias}$	guess	error	w_1 after	w_2 after	b after
1	(0,0)	0	$0 \cdot 0 + 0 \cdot 0 + 0 = 0$	1	-1	0	0	-1
1	(0,1)	0	$0 \cdot 0 + 0 \cdot 1 + (-1) = -1$	0	0 ✓	0	0	-1
1	(1,0)	0	$0 \cdot 1 + 0 \cdot 0 + (-1) = -1$	0	0 ✓	0	0	-1
1	(1,1)	1	$0 \cdot 1 + 0 \cdot 1 + (-1) = -1$	0	+1	1	1	0
2	(0,0)	0	$1 \cdot 0 + 1 \cdot 0 + 0 = 0$	1	-1	1	1	-1
2	(0,1)	0	$1 \cdot 0 + 1 \cdot 1 + (-1) = 0$	1	-1	1	0	-2
2	(1,0)	0	$1 \cdot 1 + 0 \cdot 0 + (-2) = -1$	0	0 ✓	1	0	-2
2	(1,1)	1	$1 \cdot 1 + 0 \cdot 1 + (-2) = -1$	0	+1	2	1	-1
3	(0,0)	0	$2 \cdot 0 + 1 \cdot 0 + (-1) = -1$	0	0 ✓	2	1	-1
3	(0,1)	0	$2 \cdot 0 + 1 \cdot 1 + (-1) = 0$	1	-1	2	0	-2
3	(1,0)	0	$2 \cdot 1 + 0 \cdot 0 + (-2) = 0$	1	-1	1	0	-3
3	(1,1)	1	$1 \cdot 1 + 0 \cdot 1 + (-3) = -2$	0	+1	2	1	-2
4	(0,0)	0	$2 \cdot 0 + 1 \cdot 0 + (-2) = -2$	0	0 ✓	2	1	-2
4	(0,1)	0	$2 \cdot 0 + 1 \cdot 1 + (-2) = -1$	0	0 ✓	2	1	-2
4	(1,0)	0	$2 \cdot 1 + 1 \cdot 0 + (-2) = 0$	1	-1	1	1	-3
4	(1,1)	1	$1 \cdot 1 + 1 \cdot 1 + (-3) = -1$	0	+1	2	2	-2
5	(0,0)	0	$2 \cdot 0 + 2 \cdot 0 + (-2) = -2$	0	0 ✓	2	2	-2
5	(0,1)	0	$2 \cdot 0 + 2 \cdot 1 + (-2) = 0$	1	-1	2	1	-3
5	(1,0)	0	$2 \cdot 1 + 1 \cdot 0 + (-3) = -1$	0	0 ✓	2	1	-3
5	(1,1)	1	$2 \cdot 1 + 1 \cdot 1 + (-3) = 0$	1	0 ✓	2	1	-3
6	(0,0)	0	$2 \cdot 0 + 1 \cdot 0 + (-3) = -3$	0	0 ✓	2	1	-3
6	(0,1)	0	$2 \cdot 0 + 1 \cdot 1 + (-3) = -2$	0	0 ✓	2	1	-3
6	(1,0)	0	$2 \cdot 1 + 1 \cdot 0 + (-3) = -1$	0	0 ✓	2	1	-3
6	(1,1)	1	$2 \cdot 1 + 1 \cdot 1 + (-3) = 0$	1	0 ✓	2	1	-3

Reading the trace. In epoch 1 the perceptron first says 1 to (0,0) — with all knobs at zero, $z = 0$ and “ $0 \geq 0$ ” means fire — so the bias is pushed down to -1 . Then (1,1) is wrongly answered 0, and both weights go up. In epochs 2 to 5 the rule keeps overshooting on the single-input cases (0,1) and (1,0), pulling a weight or the bias back down, then undershooting on (1,1) and pushing them up again. Slowly the bias sinks to -3 while the weights settle at 2 and 1. In epoch 6 every guess is right. **The final line is $2x_1 + x_2 - 3 = 0$ — the exact line you saw in Chapter 3.** Nobody chose it. The four examples did.



What the rule learned. AND after 6 epochs, OR after 4 — drawn from the actual runs. The third panel previews Chapter 6: four points no straight line can separate.

Your turn: OR, by hand

Same rule, same start (all zeros), same order of examples. The first row is done for you. Fill in the rest. It takes 4 epochs; the last epoch is all ticks. Then check your table in the notebook (Section 4), which prints exactly this trace.

epoch	x	y	$z = (\text{weights} \cdot \text{inputs}) + \text{bias}$	guess	error	w_1 after	w_2 after	b after
1	(0,0)	0	$0 \cdot 0 + 0 \cdot 0 + 0 = 0$	1	-1	0	0	-1
1	(0,1)	1						
1	(1,0)	1						
1	(1,1)	1						
2	(0,0)	0						
2	(0,1)	1						
2	(1,0)	1						
2	(1,1)	1						
3	(0,0)	0						
3	(0,1)	1						
3	(1,0)	1						
3	(1,1)	1						
4	(0,0)	0						
4	(0,1)	1						
4	(1,0)	1						
4	(1,1)	1						

final answer: $w_1 = 1$, $w_2 = 1$, $b = -1$ — so $z = x_1 + x_2 - 1$, which is ≥ 0 unless both inputs are 0

That is the whole learning rule — the same four moves as Book 1, done with a pencil. Two questions remain. When is the rule *sure* to finish? And what happens when it cannot? Chapter 5 answers the first; Chapter 6 the second.

5

When does a line exist? Linearly separable data

The perceptron can learn anything you can cut with one straight knife stroke — and nothing else.

∞ THINK OF IT LIKE... ONE CUT OF A KNIFE

Spread teal marbles and white marbles on a table. Now take a long ruler and try to lay it across the table so that every teal marble is on one side and every white marble on the other. If you can, the marbles are **linearly separable**. If every position of the ruler leaves some marble on the wrong side, they are not. A perceptron is the ruler.

IN ONE LINE Data is **linearly separable** when one straight line (or flat wall, in more dimensions) puts all the 1s on one side and all the 0s on the other.



Three little worlds. Left: two clouds that one line separates — a perceptron will learn this. Middle: the two groups sit on opposite diagonals; every line strands marbles. Right: one group inside the other; no line can free the middle. The perceptron can only ever draw the left picture.

NOT, AND and OR were easy because their four (or two) points are separable: in Chapter 4 you saw the lines. And when a line exists, the learning rule is not merely hopeful — it is guaranteed.

💡 ROSENBLATT'S PROMISE

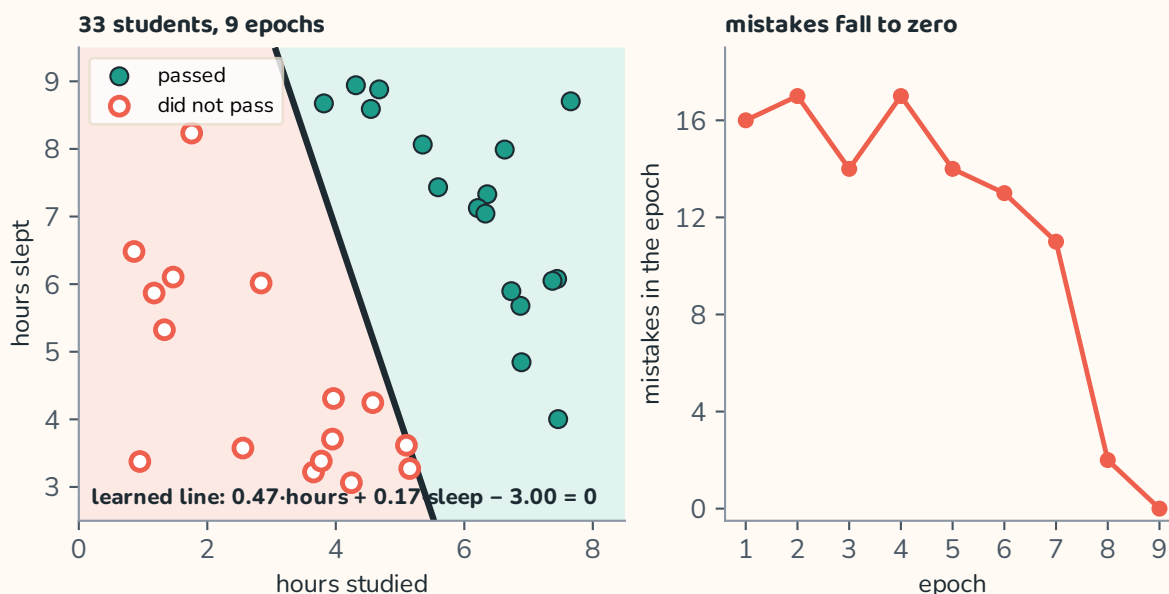
If any straight line can separate the examples, this rule will **find one in a finite number of steps** — from any starting knobs, in any order of examples. That is a theorem (proved by Novikoff in 1962), not a hope. Two small prints: it finds a line, not necessarily the nicest one; and if *no* line exists, the promise says nothing at all. Chapter 6 is about what happens then.

Let's see the promise at work on something that looks more like real data.

Thirty-three students

STORY

Asha collects two numbers from 33 classmates before a quiz — hours studied and hours slept the night before — and whether they passed. She wants a perceptron that predicts “pass” from the two numbers. Two inputs, so the perceptron will draw a line on a graph of study-hours against sleep-hours.



Learning on real-looking data. Left: the 33 students and the line the perceptron learned, after 9 epochs, starting from zero knobs. Right: mistakes per epoch. Notice the bumps — fixing one student sometimes breaks another — but because a separating line exists, the mistakes eventually hit zero and the rule stops. (Made-up data, realistic shape. Run it yourself in notebook Section 9.)

Read the learned line: $0.47 \text{ hours} + 0.17 \text{ sleep} - 3 = 0$. Both weights are positive: studying helps and sleeping helps. Studying helps about three times more per hour. A student who studies 4 hours and sleeps 7 has $z = 1.87 + 1.17 - 3 = 0.04 \rightarrow$ just passes. The perceptron has turned 33 examples into a rule of thumb, and the rule is readable.

STOP AND THINK

Only one input now: room temperature. (a) “Too hot if temperature ≥ 30 ” — can a perceptron learn this? (b) “Comfortable if the temperature is between 20 and 25” — can it?

✓ OUR ANSWER

(a) Yes: with one input the “line” is just a cut-point on the number line; $w = 1$, $b = -30$ does it. (b) **No.** “Comfortable” needs the middle piece of the number line with both ends cut off — two cuts. One perceptron makes one cut. So non-separable data exists even with a single input. Keep this example; it comes back in the next chapter.

Three things worth knowing about the line it finds

It finds a line, not *the* line. Many lines separate the same data (remember the dashed line in Chapter 3). The rule stops at the first one that makes no mistakes. Feed the examples in a different order and you may get a different line. (A later idea, the support vector machine, deliberately picks the line in the middle of the

widest gap — a different course, same picture.)

It works in any number of dimensions, unchanged. With 3 inputs the line becomes a flat plane; with 16,000 inputs, a flat wall we cannot draw. The rule $w_i \leftarrow w_i + e x_i$ does not care how many inputs there are. Rosenblatt's promise holds in every dimension.

Separability is about the data and the features you chose. The comfortable-temperature band is not separable using the temperature itself. Using the feature *distance from 22.5°* instead, it becomes one cut: “comfortable if distance ≤ 2.5 ”. Same world, different input, and the wall appears. Hold that thought.

CAREFUL — “ZERO MISTAKES” IS A STRONG DEMAND

The perceptron only stops when it is *perfect* on the training examples. Real data is never perfect: one student who slept 9 hours, studied 8, and still failed makes the data non-separable, and the rule will run forever, swinging the line around trying to save one hopeless case. Real learners must be able to say “good enough”. Notice also that the error is only ever +1, -1 or 0: a guess wrong by a whisker and one wrong by a mile get the same nudge, and a correct guess — however lucky — gets nothing. The step function has no “almost”. Book 3 replaces it with a loss that measures *how* wrong, not just *whether* wrong.

TRY IT YOURSELF — THREE SEPARABILITY PUZZLES (PAPER ONLY)

(a) Four points with two inputs: $(1,1) \rightarrow 1$, $(2,2) \rightarrow 1$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$. Plot them. Separable? If yes, write down any w_1 , w_2 , b that works and check all four.

(b) One input: $x = 0 \rightarrow 0$, $x = 1 \rightarrow 1$, $x = 2 \rightarrow 1$, $x = 3 \rightarrow 0$. Separable? (Think about the temperature band.)

(c) Three inputs, the “majority vote”: output 1 when at least two of the three inputs are 1. Write the eight examples. Find weights and a bias by reasoning, not by running the rule. Then, if you like, run the rule in the notebook and compare its answer with yours.

(a) yes — e.g. $w = (1, 1)$, $b = -1.5$ · (b) no — two cuts needed · (c) yes — e.g. $w = (1, 1, 1)$, $b = -2$ (fires when the sum of inputs is at least 2)

6

The wall called XOR

Four examples. Two inputs. A puzzle a child can state — and a single neuron can never learn. Here is why, and what it will take to get past it.

STORY

Nova learned AND in six epochs and OR in four. Asha is pleased and sets one more gate, the “exactly one” gate: output 1 when exactly one of the two inputs is 1. Nova starts. One epoch. Ten. A hundred. A thousand. The knobs keep moving and the mistakes never stop. Asha checks her code three times. The code is fine. The puzzle is not.

The puzzle

XOR (“exclusive or”) says 1 when the two inputs are *different*, and 0 when they are the same. You use it every day: a staircase light with a switch at the top and a switch at the bottom is on exactly when the two switches disagree.

Draw the four points and try to place one straight line with both 1s on one side and both 0s on the other. Really try — on paper — before reading on.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The four-line proof

“I couldn’t find a line” is not the same as “no line exists”. Let’s prove it, using nothing but the perceptron’s own rule. Suppose some weights w_1 , w_2 and bias b *did* give XOR. Then each of the four examples forces one fact:

example	wanted	so we need	in words
(0, 0)	0	$b < 0$	with both inputs off, only the bias is left, and it must be negative
(0, 1)	1	$w_2 + b \geq 0$	input 2 alone must lift the total to zero or above
(1, 0)	1	$w_1 + b \geq 0$	input 1 alone must do the same
(1, 1)	0	$w_1 + w_2 + b < 0$	but both together must fall below zero

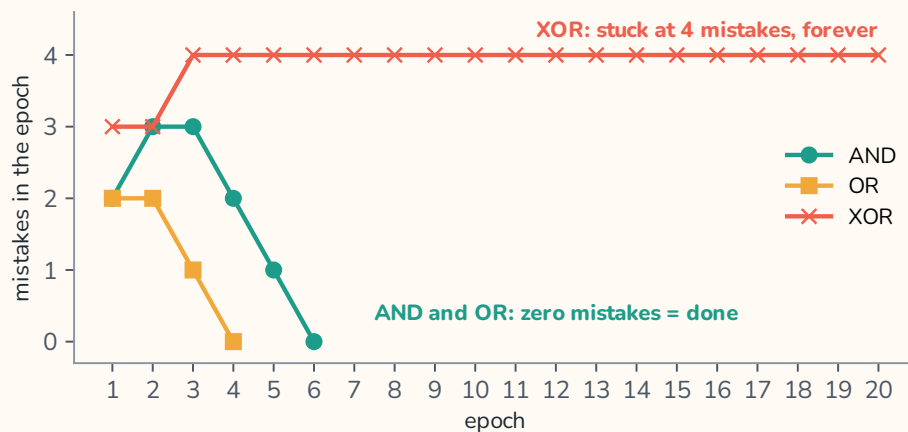
Now add the second and third facts: $w_1 + w_2 + 2b \geq 0$. Subtract b from both sides: $w_1 + w_2 + b \geq -b$. The first fact says b is negative, so $-b$ is positive. So $w_1 + w_2 + b$ is at least a positive number — it is ≥ 0 . But the fourth fact demands it be < 0 . Both cannot be true. **No such weights exist.**

💡 BIG IDEA — WHY THE PROOF WORKS

Each single input, alone, must be strong enough to fire the neuron. But then both inputs together are even stronger — a sum can only add — and the bias can only push back *once*, not twice. A single weighted sum has no way to say “either one is good, but both is bad”. It cannot do *exactly one*. No amount of training can fix an impossible job.

What the learning rule does when the job is impossible

It never stops. From epoch 2 onward the knobs go round the same four settings forever: $(w_1, w_2, b) = (-1, 0, -1) \rightarrow (-1, 1, 0) \rightarrow (0, 1, 1) \rightarrow (-1, 0, 0) \rightarrow$ back to $(-1, 0, -1)$. Each nudge fixes one example and breaks another. The mistakes-per-epoch count sits at 4 for as long as you care to wait.



Two gates finish; one never does. AND reaches zero mistakes at epoch 6, OR at epoch 4. XOR is still making 4 mistakes at epoch 20, and at epoch 20,000. Rosenblatt’s promise was conditional, and the condition — a line exists — is false.

✧ SPARK — THE WALL THAT STOPPED A FIELD

This is the example Minsky and Papert made famous in 1969. The lesson people took was “neural networks are a dead end”. The lesson they *should* have taken was on the very next page of the same book: the wall belongs to *one* neuron. Let’s climb it, twice.

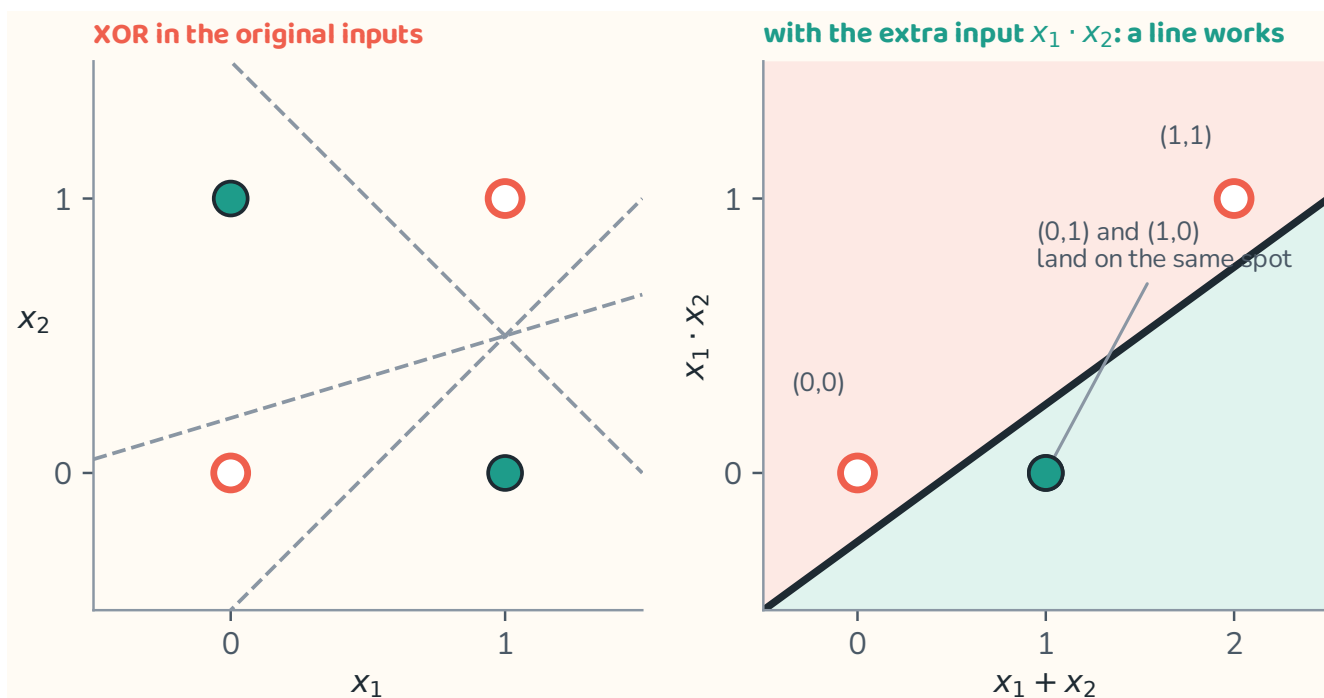
Way out 1: give the neuron a better input

Remember the comfortable-temperature band from Chapter 5: not separable with the raw temperature, separable with a smarter feature. Do the same here. Hand the neuron a **third input**, $x_3 = x_1 \cdot x_2$, which is 1 only when both inputs are 1. Now the four examples live in three numbers, and a flat wall *can* separate them:

x_1	x_2	$x_1 \cdot x_2$	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Run the same learning rule on these three-input examples and it converges in 8 epochs to $\mathbf{w} = (1, 1, -3)$, $\mathbf{b} = -1$. Check: $(1,1,1)$ gives $1 + 1 - 3 - 1 = -2 \rightarrow 0$; $(0,1,0)$ gives $0 + 1 + 0 - 1 = 0 \rightarrow 1$. The extra input lets the neuron say “both is bad”: the -3 knob on $x_1 \cdot x_2$ cancels the two $+1$ s.

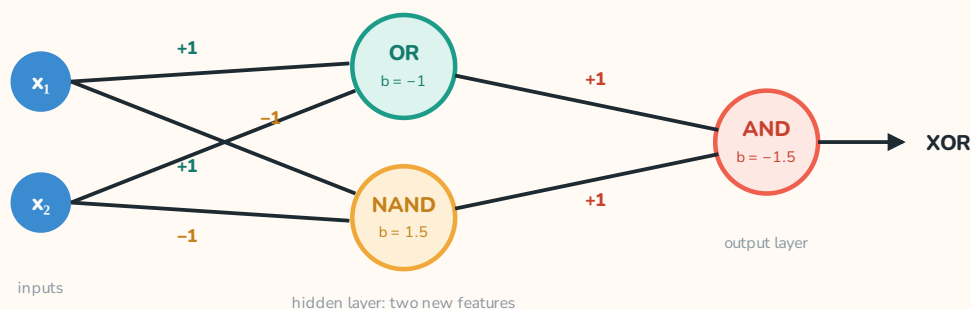
This is the “old way” from Book 1: a human invents the right feature, a simple learner does the rest. It works — when a human can find the feature.



A new axis makes the wall appear. Left: XOR in its original inputs; no line. Right: the same four examples plotted by $(x_1 + x_2)$ and $(x_1 \cdot x_2)$. The two 1s land on the same spot, and one line now separates them from the two 0s.

Way out 2: a second layer

Now the idea that runs the rest of this course. Instead of inventing a feature, let *other neurons* build one. XOR can be written with gates we already have: **XOR = $(x_1 \text{ OR } x_2) \text{ AND NOT}(x_1 \text{ AND } x_2)$** — “at least one, but not both”. So put two neurons in a first layer, an OR and a NAND (“not both”), and let one last neuron take an AND of *their* answers.



Three perceptrons, two layers, XOR solved. The OR neuron (weights +1, +1, bias -1) fires when at least one input is on. The NAND neuron (weights -1, -1, bias 1.5) fires unless both are on. The AND neuron (weights +1, +1, bias -1.5) fires only when both hidden neurons fire.

x_1	x_2	OR fires?		NAND fires?		AND of those		wanted
0	0	0	(-1)	1	(1.5)	0	(-0.5)	0 ✓
0	1	1	(0)	1	(0.5)	1	(0.5)	1 ✓
1	0	1	(0)	1	(0.5)	1	(0.5)	1 ✓
1	1	1	(1)	0	(-0.5)	0	(-0.5)	0 ✓

The small numbers are each neuron's total z . Trace one row yourself: for (1, 1), OR gets $1 + 1 - 1 = 1 \rightarrow$ fires; NAND gets $-1 - 1 + 1.5 = -0.5 \rightarrow$ quiet; AND gets $1 \cdot 1 + 1 \cdot 0 - 1.5 = -0.5 \rightarrow$ quiet. Output 0. Correct.

💡 BIG IDEA — WHAT THE HIDDEN LAYER REALLY DID

Look at the middle two columns of the table. The hidden neurons turned the four inputs into four new descriptions: (0,1), (1,1), (1,1), (1,0). In this new description the 1s sit together and the 0s sit apart — the last neuron only needs one cut. **The hidden layer re-described the problem until a line was enough.** That is Book 1's "layers learn features", seen up close for the first time. Way out 1 needed a human to invent $x_1 \cdot x_2$. Way out 2 lets neurons invent it.

⚠️ CAREFUL — THE CATCH, AND THE CLIFFHANGER

We set the hidden weights *by hand*. Can the learning rule set them? No — and the reason is worth stating exactly. The rule needs an error, $y - \hat{y}$, for each neuron it adjusts. The final neuron has one: Asha says whether XOR came out right. But nobody ever tells the OR neuron what *it* should have said. Its correct answer is not in the data. This is called the **credit assignment problem**: when the team gets it wrong, which player should change? Worse, the step function passes back no information at all — a hidden neuron's output is a bare 0 or 1, with no hint of "nearly".

Solving this needs two things we do not yet have: a **smooth neuron** whose output can say "nearly" (Books 3 and 4), and a way to **pass blame backwards** through the layers (Book 5, backpropagation). That is the route from here to a network that can hear "Hey Nova".

💡 STOP AND THINK

In Chapter 3 you were asked to make Nova wake up on *exactly one* of the two words. What have you now learned about that task? And is Nova's real job — 16,000 raw numbers in, wake-word out — more like AND, or more like XOR?

✓ OUR ANSWER

"Exactly one word" is XOR, so no single perceptron can do it — you were set an impossible task on purpose. As for the real job: on raw sound, the "Hey Nova" clips form a twisted, scattered region in a 16,000-dimensional space, far more tangled than the ring in Chapter 5. No single wall separates them. Nova needs layers to re-describe the sound until a wall is enough — which is exactly what Book 1's stack of layers was doing.

The demo: a perceptron in twenty lines

Everything you did with a pencil, a computer can do in a blink — and it will print exactly the same tables.

The companion notebook, `Book2_Perceptron_From_Scratch.ipynb`, contains all the code below and runs it. Nothing is imported except NumPy (for arrays) and Matplotlib (for pictures). No library does the learning for us — that is the whole point. Read the code the way you read the trace tables: line by line, checking that it does what Chapter 4 says.

1 • The neuron and its rule

```
import numpy as np

def step(z):
    # the decision: fire (1) if the total is 0 or more, else stay quiet (0)
    return 1 if z ≥ 0 else 0

class Perceptron:
    def __init__(self, n_inputs, lr=1.0):
        self.w = np.zeros(n_inputs)    # one knob per input, all start at 0
        self.b = 0.0                    # the bias knob
        self.lr = lr                    # learning rate: how big a nudge

    def total(self, x):
        # z = w1*x1 + w2*x2 + ... + b
        return float(np.dot(self.w, x) + self.b)

    def predict(self, x):
        return step(self.total(x))

    def train(self, X, y, max_epochs=20):
        mistakes_per_epoch = []
        for epoch in range(1, max_epochs + 1):
            mistakes = 0
            for xi, yi in zip(X, y):    # one example at a time
                guess = self.predict(xi) # 1 • guess
                err = yi - guess         # 2 • measure: +1, 0 or -1
                if err ≠ 0:              # 3 • nudge, only on a mistake
                    self.w = self.w + self.lr * err * np.array(xi)
                    self.b = self.b + self.lr * err
                    mistakes += 1
            mistakes_per_epoch.append(mistakes)
            if mistakes == 0:           # 4 • repeat until a clean epoch
                break
        return mistakes_per_epoch
```

Match the four comments to the four steps of Rosenblatt’s rule in Chapter 4. The line `self.w + self.lr * err * np.array(xi)` is $\mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x} \rightarrow w_i \leftarrow w_i + \eta e x_i$ for all inputs at once; NumPy multiplies the whole list in one go. That one line is the entire “learning”.

2 • The gates, and running them

```
GATES = {
    "NOT": ([[0], [1]], [1, 0]),
    "AND": ([[0,0], [0,1], [1,0], [1,1]], [0, 0, 0, 1]),
    "OR": ([[0,0], [0,1], [1,0], [1,1]], [0, 1, 1, 1]),
    "XOR": ([[0,0], [0,1], [1,0], [1,1]], [0, 1, 1, 0]),
}
for name, (X, y) in GATES.items():
    p = Perceptron(n_inputs=len(X[0]))
    m = p.train(X, y, max_epochs=20)
    print(name, "epochs:", len(m), "mistakes per epoch:", m, "w =", p.w, "b =", p.b)
```

And the output — compare every number with your paper work:

```
NOT  epochs: 3  mistakes per epoch: [1, 1, 0]      w = [-1.]      b = 0.0
AND  epochs: 6  mistakes per epoch: [2, 3, 3, 2, 1, 0]  w = [2. 1.]    b = -3.0
OR   epochs: 4  mistakes per epoch: [2, 2, 1, 0]      w = [1. 1.]    b = -1.0
XOR  epochs: 20 mistakes per epoch: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
```

Three gates finish with a clean epoch. XOR hits the 20-epoch limit still making 4 mistakes — raise the limit to 20,000 and nothing changes. The notebook version of `train` also prints the full trace, one line per example, which is how the tables in Chapter 4 were made.

3 • What else is in the notebook

Section	What it does	Book chapter
2 – 4	Full traces for NOT and AND; OR left for you to predict first, then check.	Chapter 4
5	Draws the learned lines for AND and OR, with the four points and the two shaded sides.	Chapters 3 and 4
6	Runs XOR for 6 epochs so you can see the knobs go round in a circle; plots mistakes per epoch for all three gates.	Chapter 6
7	Way out 1: adds the input $x_1 \cdot x_2$ and trains again — converges in 8 epochs.	Chapter 6
8	Way out 2: the OR + NAND → AND network with hand-set weights, printing the full truth table.	Chapter 6
9	The 33 students: a perceptron on data with real-valued inputs, plus the picture of the learned line.	Chapter 5
10	Six things to try: NAND and NOR by hand; change the order of the examples; change the learning rate (the epochs do not change — work out why); a 3-input AND; break separability on purpose and watch the mistakes never reach zero.	everything

Look back, look ahead

The whole book in six sentences, a self-check, questions worth carrying, and where Book 3 begins.

The story in six sentences

1. A real neuron **adds up** what its neighbours say, each weighted by a synapse, and **fires** if the total crosses a threshold; learning is synapses changing strength.
2. The artificial neuron keeps only that arithmetic: $z = \sum_i w_i x_i + b$, **output 1 if $z \geq 0$** — for a whole table of examples, $\mathbf{z} = \mathbf{X}\mathbf{w} + b$. Connectionism says the program *is* the weights.
3. A perceptron with two inputs is a **straight line and a side**; with more inputs, a flat wall. The weights set the direction, the bias sets where it stands.
4. **Rosenblatt's rule** — $\mathbf{w} \leftarrow \mathbf{w} + (y - \hat{y})\mathbf{x}$, $b \leftarrow b + (y - \hat{y})$ — nudges only the wires that were on, only when the guess was wrong, and finds a separating line whenever one exists.
5. It can therefore learn exactly the **linearly separable** problems: NOT, AND, OR, the 33 students — anything one knife stroke can cut.
6. **XOR** cannot be cut by one line (a four-line proof), so one neuron can never learn it; two layers can — but the rule cannot train the hidden layer, and that is the problem the rest of the course solves.

Explain it to a friend

Cover the right column. Answer out loud, in your own words. Then compare.

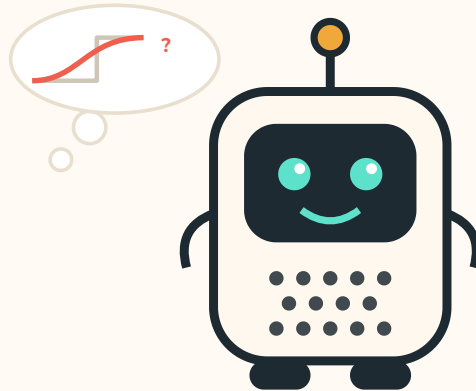
Question	A good short answer
What do a real neuron and an artificial one share?	Weighted inputs, a sum, a threshold — and weights that can change. Everything else about the real one is thrown away.
What does the bias do?	It sets how easy it is to fire. Geometrically, it slides the line without turning it.
Why multiply the nudge by x_i ?	An input that was 0 contributed nothing, so it gets no blame; only the wires that were on get adjusted.
Why does the AND perceptron end with $b = -3$?	Because both inputs, each with weight 2 or 1, must be on to reach 0: $2 + 1 - 3 = 0$. Either alone falls short.
What is “linearly separable”?	One straight line (or flat wall) can put all the 1s on one side and all the 0s on the other.
Why can't one perceptron do XOR?	Each input alone must lift the sum to 0 or more, so both together lift it even higher; the bias can push back only once. Sums can't say “either but not both”.
What does a hidden layer add?	New features. It re-describes the input until the last neuron only needs one cut.
Why can't Rosenblatt's rule train the hidden layer?	Nobody tells a hidden neuron what it should have output, so there is no error to nudge with — the credit assignment problem.

✦ QUESTIONS WORTH CARRYING (NO ANSWERS HERE — ON PURPOSE)

- 1 · The perceptron's weights for the 33 students were readable: "studying helps three times more than sleeping". Nova's network will have millions of weights. Will they still mean anything? Should we care?
- 2 · The rule only moves when there is a mistake, and always by the same amount. Design, in words, a rule that moves *more* when the guess was *more* wrong. What would you need the neuron to output for that to be possible?
- 3 · Way out 1 (invent a feature) and way out 2 (add a layer) solved XOR equally well. When would you choose each? What does a human have to know in each case?
- 4 · Hebb: "fire together, wire together." Rosenblatt: "nudge the active wires on a mistake." What does the second rule have that the first lacks? Could Hebb's rule alone learn AND?

Words to know — the whole book

Neuron (biological) A cell: dendrites listen, cell body adds, axon fires if the total crosses a threshold.	Synapse A connection between neurons; strong or weak; changes with experience.	Artificial neuron $z = \sum_i w_i x_i + b$, then a threshold. The cartoon of the real thing.
Connectionism Intelligence from many simple connected units; knowledge stored in the weights.	Perceptron One artificial neuron with a step output, used as a yes/no classifier.	Weight The knob on one input: sign = for/against, size = how much it matters.
Bias The extra knob that sets how easy firing is; slides the line.	Threshold θ The old way to write the bias: fire if $\sum w \cdot x \geq \theta$, with $\theta = -b$.	Step function Output 1 if $z \geq 0$, else 0. All-or-nothing.
Decision boundary The line (or wall) where $z = 0$; the perceptron answers "which side?".	Hyperplane A flat wall in many dimensions — the decision boundary with many inputs.	Learning rule (Rosenblatt) $\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$, $b \leftarrow b + \eta(y - \hat{y})$, one example at a time.
Error $y - \hat{y}$: +1, 0 or -1 for the perceptron.	Learning rate η How big each nudge is.	Epoch One pass through all the examples.
Convergence Reaching an epoch with zero mistakes — guaranteed if a separating line exists.	Linearly separable One straight line/wall can split the 1s from the 0s.	XOR "Exactly one": output 1 when the inputs differ. Not linearly separable.
Feature An input to the neuron; a good one (like x_1, x_2) can make a wall appear.	Hidden layer Neurons between input and output whose job is to build new features.	Credit assignment Deciding which hidden neuron to blame when the final answer is wrong.



End of Book 2

Nova can now learn anything one straight line can cut. It is wondering what to do about the corners.

NEXT TIME — BOOK 3: GUESSING “HOW MUCH”

The perceptron’s step function is a cliff: 0 or 1, never “almost”. That is why its learning rule is deaf to *how* wrong a guess was, and why it can only answer yes/no. In Book 3 we throw the cliff away. One neuron will output a plain number — a car’s fuel mileage, predicted from its weight and engine — and we will score it with the squared error from Book 1.

Then, for the first time, we will do the maths of walking downhill: how a computer can ask every knob “which way lowers the loss?” and take a small step. That method, **gradient descent**, is the optimizer that will train everything for the rest of the course — including, in Book 5, the hidden layer that Rosenblatt’s rule could not reach.

Deep Learning, Told Simply · a five-book story for the Deep Learning (DNN) course · Prof. Saurabh